

Lecture Handout: Securing Applications with OpenID Connect (OIDC)

One of the selling points of a web framework like ASP.NET Core is the ability to provide a dynamic app, customized to individual users. Many apps have the concept of an “account” with the service, which you can “sign in” to and get a different experience. Depending on the service, an account gives you varying things: on some apps you may have to sign in to get access to additional features, on others you might see suggested articles. On an e-commerce app, you’d be able to make and view your past orders, on Stack Overflow you can post questions and answers, whereas on a news site you might get a customized experience based on previous articles you’ve viewed. When you think about adding users to your application, you typically have two aspects to consider:

1. Core Concepts: Authentication vs. Authorization

Security in any application requires two distinct mechanisms:

- **Authentication:** Identifying the user attempting to perform actions. It is the process of creating users and letting them log in to your app. (who you are)
- **Authorization:** Allowing specific privileges or actions to be given to an already authenticated user. This also allows customizing the experience and controlling what users can do, based on the current logged-in user. (what you’re allowed to do)

2. OAuth 2.0 : Is an authorization protocol. A **protocol** (in computing and networking) is simply a **set of rules that define how systems communicate with each other**.

OAuth 2.0 Flow (Simplified)

1. User clicks **Login with (Google, Facebook,)**
2. Client redirects user to authorization server
3. User logs in & gives consent
4. Authorization server returns **authorization code**
5. Client exchanges code for **access token**
6. Client uses token to access API (resource server)

3. Introduction to OpenID Connect (OIDC)

OpenID Connect (OIDC) is an identification standard built on top of the **OAuth 2.0** authorization protocol. Its primary goal is to **delegate user authentication** to a trusted third-party service, such as Google, Facebook, Apple, or Microsoft.

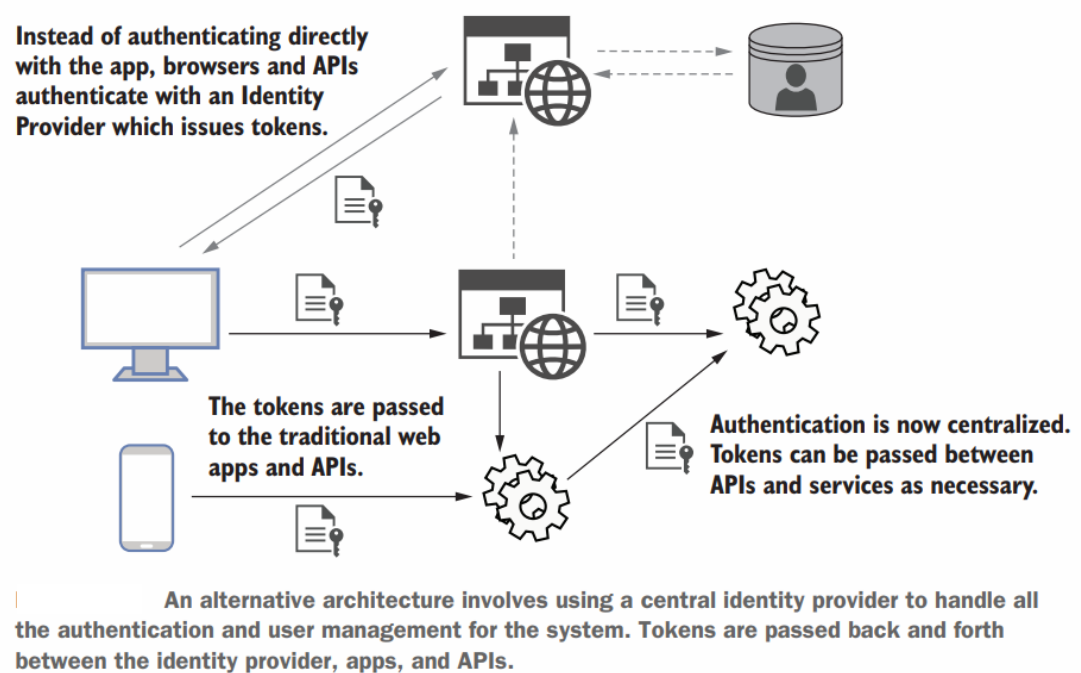
This follows the **Single Sign-On (SSO)** principle, where a single authentication system is reused across multiple applications. The interaction involves **three actors**:

1. **The Client:** The application (e.g., a web app).
2. **The Identity Provider (IdP):** The service that verifies the user's identity.
3. **The Protected Resource:** The API or data being accessed.

4. The JSON Web Token (JWT)

When a user authenticates with the IdP, the provider issues a **JSON Web Token (JWT)**.

- **Structure:** A JWT is encoded in **Base64** and consists of **three** parts separated by **dots**: **Header**, **Content (Payload)**, and **Signature**.
- **Validation:** The application validates the token **autonomously** by retrieving metadata from the identity provider only once. It uses a key to verify the signature, ensuring the token was issued by the correct provider and has not been tampered with.
- **Claims:** The content of the token contains "claims," which are key-value pairs defining the user's identity, email, and assigned **roles**.



5. Configuring ASP.NET Core Security

To secure an API, you must configure the server to recognize and trust the JWTs issued by your chosen IdP.

- A. Installation:** Install the NuGet package:
Microsoft.AspNetCore.Authentication.JwtBearer.

Tools → NuGet Package Manager → Package Manager Console

Then write “Install-Package Microsoft.AspNetCore.Authentication.JwtBearer” in the console window. Check that Dependencies → Packages contains the package.

Then to use it add using Microsoft.AspNetCore.Authentication.JwtBearer; to your Program.cs code.

B. Configuration (Program.cs): You must define how the application validates incoming tokens:

- **AddAuthentication:** Defines the authentication scheme (usually JwtBearer).
- **AddJwtBearer:** Sets the parameters for trust:
 - **Authority:** The URL of the identity provider.
 - **Audience:** The unique identifier for your application.
 - **ValidateLifetime & ValidateIssuer:** Set to true to ensure the token is current and from the right source.
 - **ClockSkew:** Manages the time gap (usually **5 minutes**) between the IdP's clock and your server's clock to prevent accidental rejection of valid tokens.

C. The Middleware Pipeline: The order of middleware is critical. UseCors must be placed **before** authentication and authorization:

1. app.UseCors();
2. app.UseAuthentication(); (Registers the auth middleware).
3. app.UseAuthorization(); (Activates authorization rules).

6. Protecting Endpoints and Identifying Users

- **Restricting Access:** Use the .RequireAuthorization() extension method on minimal endpoints to ensure only authenticated users can access them.
- **Role-Based Security:** You can define **policies** (e.g., "SurveyCreator") that require a user to have a specific role claim within their JWT to access a resource.
- **Handling Errors:**
 - **401 Unauthenticated:** The user does not have a valid JWT.
 - **403 Forbidden:** The user is authenticated but lacks the necessary roles/permissions.

- **Accessing Identity:** You can inject the ClaimsPrincipal class into your endpoints to read the user's claims directly from the token. For cleaner code, you can design a UserProfile service that uses IHttpContextAccessor to expose user data like names and roles.

7. Passing the Token

For every request to a protected resource, the client must pass the JWT in the HTTP header as follows:

Authorization: bearer {YourJWT}